

IRAC/CREAC Structural Highlighter

Project Specification — v1.0

Date: April 11, 2026 **Status:** Ready for Claude Code handoff **Prepared by:** Anthony (UBalt Law, J.D. Candidate 2028)

1. Project Overview

The IRAC/CREAC Structural Highlighter is a fully offline Python desktop application that helps law students identify and correct structural deficiencies in legal writing drafts (practice exams, memos). The app ingests a draft document, classifies each sentence according to either the IRAC or CREAC framework using local heuristic analysis, and produces a color-coded, annotated view with structural problem flags and fix suggestions. An annotated `.docx` export is available for download.

2. Goals and Non-Goals

Goals (v1.0)

- Accept drafts via plain text paste, `.docx` upload, or `.pdf` upload
- Classify sentences as IRAC or CREAC structural components using offline heuristics
- Display sentence-level color highlighting and paragraph-level summary badges
- Flag structural problems and provide plain-English fix suggestions
- Export an annotated `.docx` copy with color highlights matching the UI

Non-Goals (explicitly out of scope for v1.0)

- AI/LLM-powered classification (no API calls; fully offline)
 - Manual label correction by the user in the UI (deferred to v2)
 - Support for real-world briefs or non-law-school writing contexts
 - Support for frameworks other than IRAC and CREAC
 - `.pdf` export (only `.docx` export is required)
 - Multi-document batch processing
-

3. Architecture and Delivery Format

Recommended Stack

Layer	Technology
Backend / server	Python 3.10+, Flask
Frontend	HTML + vanilla JavaScript (served by Flask)
Document parsing	<code>python-docx</code> (.docx), <code>pdfplumber</code> (.pdf)
Document export	<code>python-docx</code>
Sentence tokenization	<code>nltk</code> (punkt tokenizer) or regex fallback
Packaging / launch	Single <code>run.py</code> launcher; opens browser automatically

Runtime Model

The app runs as a local Flask server on `localhost:5050` (or next available port). On launch, it opens the user's default browser to the app UI. No internet connection is required at any point. There are no external API calls. The app can be run from a terminal with `python run.py`.

Directory Structure (suggested)

```
irac-highlighter/
├─ run.py                # Entry point; starts Flask and opens browser
├─ app/
│   ├── __init__.py
│   ├── routes.py        # Flask routes
│   ├── classifier.py     # Core heuristic classification engine
│   ├── parser.py        # Document ingestion (.docx, .pdf, plaintext)
│   ├── exporter.py      # Annotated .docx export
│   └─ templates/
│       ├── index.html   # Single-page UI
│       └─ static/
│           ├── style.css
│           └─ app.js
├─ requirements.txt
└─ README.md
```

4. Supported Frameworks

4.1 IRAC

Component	Label	Color
Issue	ISSUE	Pink /  #EC4899
Rule	RULE	Green /  #22C55E
Application	APPLICATION	Orange /  #F97316
Conclusion	CONCLUSION	Blue /  #3B82F6

4.2 CREAC

Component	Label	Color
Conclusion (opening)	CONCLUSION	Blue /  #3B82F6
Rule	RULE	Green /  #22C55E
Explanation	EXPLANATION	Purple /  #A855F7
Application	APPLICATION	Orange /  #F97316
Conclusion (closing)	CONCLUSION	Blue /  #3B82F6

Note: The color palette must be accessible (WCAG AA contrast against white background). Each sentence span in the rendered output receives a semi-transparent background highlight using these colors.

5. Input Formats

The app accepts three input methods, selectable via the UI:

Method	Implementation
Plain text paste	<code><textarea></code> in UI; text sent directly to backend
<code>.docx</code> upload	File input; parsed with <code>python-docx</code> ; paragraphs extracted as text
<code>.pdf</code> upload	File input; parsed with <code>pdfplumber</code> ; text extracted page by page

Parsing requirements:

- Preserve paragraph breaks; do not collapse multi-paragraph documents into a single block.
- Strip headers, footers, and page numbers from `.docx` and `.pdf` inputs where detectable.
- Normalize whitespace (collapse multiple spaces, strip leading/trailing whitespace per sentence).
- For `.pdf`, if text extraction yields fewer than 50 characters per page on average, display a warning: `"This PDF may be scanned or image-based. Text extraction may be incomplete."`

6. Classification Engine

6.1 Architecture

The classifier operates fully offline using a rule-based heuristic system. No machine learning models, embeddings, or external APIs are used.

Pipeline:

1. Tokenize input text into sentences (using `nltk` punkt tokenizer; fallback to period-based split)
2. Group sentences into paragraphs (preserve original paragraph breaks from the document)
3. For each sentence, apply heuristic rules to assign a structural label
4. For each paragraph, derive a paragraph-level summary and flag structural problems

6.2 Heuristic Rules

Rules are evaluated in priority order. The first matching rule wins.

Issue Detection

A sentence is labeled `ISSUE` if it matches any of the following:

- Contains phrases: "the issue is", "the question is", "the question presented", "the dispositive question", "at issue is", "whether" (when appearing as first word or after a comma near sentence start), "the dispute", "the problem is"
- Is an interrogative sentence (ends with ?)
- Is the first sentence of the document or of a major section header

Rule Detection

A sentence is labeled **RULE** if it matches two or more of the following signals:

- Contains a legal citation marker: case name patterns (v., v, v.), statute markers (§, U.S.C., C.F.R., Restatement), or Bluebook-style citation patterns (two or more capital words followed by a number)
- Contains legal standard phrases: "under [the] [rule/law/statute/doctrine]", "courts have held", "courts require", "the rule is", "a [plaintiff/defendant/party] must [show/establish/prove/demonstrate]", "elements of", "the standard [is/for]", "is defined as", "requires [that]", "is established when", "in order to" + legal verb, "pursuant to", "provides that", "prohibits", "the [Act/Code/statute] [states/requires/mandates]"
- Does **not** contain application trigger words (see Application Detection below)
- Does not refer to named parties from the fact pattern (if party names can be inferred from context)

Explanation Detection (CREAC only)

A sentence is labeled **EXPLANATION** if CREAC mode is active and it matches:

- Contains phrases referencing precedent reasoning: "in [case name], " (followed by a description), "the court [found/determined/concluded/noted/observed] that", "for example", "as in", "similarly, in", "by contrast, in", "the [plaintiff/defendant] in that case", "that court", "the [majority/dissent] reasoned"
- Also contains a citation marker (to distinguish from pure Application sentences)

Application Detection

A sentence is labeled **APPLICATION** if it matches any of the following:

- Begins with or contains: "here, ", "here the", "in this case, ", "in the present case, ", "in our case, ", "under these facts, ", "on these facts, ", "applying

[this/the] rule", "applying [this/the] [standard/test]", "because [party name]", "since [party name]", "the facts [show/indicate/demonstrate]", "[party] [did/failed/acted/argued/contended]", "[party]'s [action/conduct/behavior]", "[party] will [argue/contend]", "this [conduct/action/behavior]"

- Contains first-person or direct-analysis language applied to the specific fact pattern rather than a general rule

Conclusion Detection

A sentence is labeled **CONCLUSION** if it matches any of the following:

- Contains: "therefore,", "thus,", "accordingly,", "for these reasons,", "for the foregoing reasons,", "in conclusion,", "in sum,", "in summary,", "as a result,", "it follows that", "a court would likely [find/hold/conclude]", "the court [should/will likely/is likely to]", "[party] is likely to [prevail/succeed/fail/be liable]", "[party] has [established/failed to establish]"
- Is the final sentence of a paragraph AND contains a hedge word (likely, probably, may, should)

Unclassified Fallback

If no rule matches, the sentence is labeled **UNCLASSIFIED** and displayed with a light gray background. Unclassified sentences are noted but not flagged as errors.

6.3 Sentence-Level Conflict Detection (Blended Sentences)

After primary classification, apply a secondary pass to detect **blended sentences** — sentences that contain signals from two incompatible categories:

- **Rule/Application Blend:** Sentence classified as **RULE** but also contains an application trigger word (here, in this case, etc.) — flag as **BLEND: RULE+APPLICATION**
- **Application without citation that looks like a Rule:** Sentence classified as **APPLICATION** but contains a citation — flag as **BLEND: RULE+APPLICATION**

Blended sentences receive a red underline (dashed) in addition to their background color.

7. Paragraph-Level Analysis

After sentence classification, each paragraph receives a summary badge displayed inline above the paragraph.

7.1 Badge Types

Badge	Trigger Condition
✅ IRAC Complete	Paragraph contains at least one sentence from each of: ISSUE (or implied), RULE, APPLICATION, CONCLUSION
✅ CREAC Complete	Paragraph contains at least one sentence from each of: CONCLUSION, RULE, EXPLANATION, APPLICATION, CONCLUSION
⚠️ Missing Rule	No RULE sentence found; APPLICATION sentences present
⚠️ Missing Application	RULE sentences present; no APPLICATION sentence found
⚠️ Missing Conclusion	No CONCLUSION sentence found
⚠️ Rule/Application Blend Detected	One or more BLEND flags present in the paragraph
⚠️ Application Without Rule	APPLICATION sentences appear before any RULE sentence in the paragraph
i Introductory / Transition	Paragraph is very short (1-2 sentences) and contains no classifiable structure — likely a roadmap or transition paragraph (not flagged as an error)
? Mostly Unclassified	More than 60% of sentences are UNCLASSIFIED

Badges are color-matched to their severity: green for complete, yellow for warnings, gray for informational.

7.2 Paragraph Problem Flags and Suggestions

Each paragraph with a warning badge displays a collapsible **Suggestions** panel directly below the badge. Suggestions are plain-English and actionable:

Problem	Suggested Fix Text
Missing Rule	"This paragraph applies facts but does not state the governing legal rule. Add a rule statement citing the applicable statute, case, or doctrine before your application."
Missing Application	"This paragraph states a rule but does not apply it to the facts. Add a sentence beginning with 'Here,' or 'In this case,' connecting the rule to the specific facts."
Missing Conclusion	"This paragraph does not reach a conclusion. Add a closing sentence stating the likely legal outcome (e.g., 'Therefore, [party] is likely to ...')."
Rule/Application Blend	"One or more sentences appear to mix a legal rule statement with fact-specific application. Consider splitting the sentence: state the rule abstractly first, then apply it to the facts in a separate sentence."
Application Without Rule	"Application language appears before the rule is stated. IRAC/CREAC requires that you state the governing rule before applying it to the facts."

8. UI / UX Requirements

8.1 Layout

The UI is a single-page application. Recommended layout:

IRAC/CREAC Structural Highlighter
[Framework: IRAC ▼] [Analyze] [Clear] [Export ▼]

INPUT PANEL

[Paste Text] [Upload .docx] [Upload .pdf]

<textarea or file drop zone>

RESULTS PANEL (appears after analysis)

LEGEND: [ISSUE] [RULE] [EXPLANATION] [APPLICATION]
[CONCLUSION] [UNCLASSIFIED] [~~BLEND~~]

[⚠ Missing Rule] [▼ Suggestions]

[ISSUE] The question is whether Acme Corp...

[APPLICATION] Here, the defendant...

[CONCLUSION] Therefore, a court would...

[✅ IRAC Complete]

[RULE] Under the Restatement (Second)...

[APPLICATION] In this case, the plaintiff...

SUMMARY BAR

Paragraphs analyzed: 8 | Complete: 3 | Warnings: 4

Sentences classified: 42 | Blends detected: 2

8.2 UI Behaviors

- **Framework selector:** Dropdown with **IRAC** and **CREAC**. Switching framework clears results and re-analyzes if text is present.
- **Input tabs:** Three tabs — **Paste Text**, **Upload .docx**, **Upload .pdf**. Switching tabs clears the other inputs.
- **Analyze button:** Sends text/file to Flask backend for processing; displays results below.

- **Legend:** Always visible in the results panel. Clicking a legend item toggles visibility of that label class (e.g., hide all RULE sentences).
- **Suggestions panel:** Collapsed by default; expands on click. Shows fix text per paragraph.
- **Sentence tooltips:** Hovering over a highlighted sentence shows a tooltip with the label (`(RULE)`, `(APPLICATION)`, etc.) and the triggering keyword or phrase that produced the classification.
- **Summary bar:** Appears at bottom of results; shows aggregate statistics.
- **Scroll sync:** Results panel scrolls independently; input panel stays visible.
- **Responsive layout:** Must be usable at 1280px width minimum. Mobile is not required.

8.3 Color and Accessibility

- All highlight colors use semi-transparent backgrounds (`(rgba)`, ~25% opacity) to preserve text readability.
 - Blended sentences use a red dashed underline, not a background color change.
 - Unclassified sentences use a light gray background (`(#F3F4F6)`).
 - The app must pass WCAG AA contrast for all displayed text on highlighted backgrounds.
-

9. Export

9.1 Annotated .docx Export

The export function produces a `(.docx)` file using `(python-docx)`. The exported document:

- Contains all analyzed text, preserving paragraph structure
- Each sentence is wrapped in a `(Run)` with a background highlight color matching its label (using `(python-docx)`'s `(font.highlight_color)` or character shading)
- Each paragraph is preceded by a bold paragraph-level summary label (e.g., `(⚠ Missing Rule)`) in the corresponding badge color
- Blended sentences receive a red font color in addition to their highlight

Color mapping for .docx highlights (WD_COLOR_INDEX):

| Label | .docx Highlight Color | Note |
|--------------|--------------------------|---|
| ISSUE | PINK | |
| RULE | BRIGHT_GREEN | |
| EXPLANATION | VIOLET | |
| APPLICATION | DARK_YELLOW | Word has no true orange; DARK_YELLOW (mustard) is the closest available value |
| CONCLUSION | TURQUOISE | Closest available to blue in Word's highlight palette |
| UNCLASSIFIED | (no highlight) | |
| BLEND | (red font, WD_COLOR.RED) | |

9.2 Export Trigger

Export is triggered by clicking `Export ▼` → `Download Annotated .docx`. The download is initiated from the Flask backend via a `/export` route that returns the `.docx` as a file attachment.

10. Flask Routes

| Route | Method | Description |
|-----------------------|--------|---|
| <code>/</code> | GET | Serve the main UI |
| <code>/analyze</code> | POST | Accepts JSON <code>{text, framework}</code> or multipart form with file + framework; returns JSON classification result |
| <code>/export</code> | POST | Accepts JSON classification result; returns <code>.docx</code> file as download |

`/analyze` Response Schema

json

```

{
  "framework": "IRAC",
  "paragraphs": [
    {
      "paragraph_index": 0,
      "badge": "WARNING_MISSING_RULE",
      "badge_label": "⚠ Missing Rule",
      "suggestion": "This paragraph applies facts but does not state...",
      "sentences": [
        {
          "text": "Here, the defendant failed to...",
          "label": "APPLICATION",
          "color_hex": "#22C55E",
          "blend": false,
          "trigger_phrase": "Here,"
        }
      ]
    }
  ],
  "summary": {
    "total_paragraphs": 8,
    "complete_paragraphs": 3,
    "warning_paragraphs": 4,
    "total_sentences": 42,
    "blend_count": 2,
    "label_counts": {
      "ISSUE": 5,
      "RULE": 12,
      "APPLICATION": 18,
      "CONCLUSION": 5,
      "EXPLANATION": 0,
      "UNCLASSIFIED": 2
    }
  }
}

```

11. Dependencies

```

flask>=3.0
python-docx>=1.1

```

```
pdfplumber>=0.11
nltk>=3.8
```

NLTK setup note: On first run, the app should automatically download the `punkt` tokenizer data if not present:

```
python

import nltk
nltk.download('punkt', quiet=True)
```

12. Error Handling

| Scenario | Behavior |
|--|--|
| Empty input | Display inline error: "No text was provided. Please paste text or upload a file." |
| Unsupported file type | Display inline error: "Unsupported file type. Please upload a .docx or .pdf file." |
| .pdf with no extractable text | Display warning banner (see Section 5) and proceed with whatever text was extracted |
| .docx parse failure | Display inline error: "Could not read this .docx file. It may be corrupted or use an unsupported format." |
| Classification yields zero labeled sentences | Display notice: "No structural markers were detected. Try selecting a different framework or verify the document contains legal analysis." |
| Port 5050 already in use | Increment port and retry up to port 5060; display in terminal which port is in use |

13. Out of Scope / Future Roadmap (v2+)

The following features are explicitly deferred and should not be implemented in v1.0:

- Manual sentence label correction by the user

- AI/LLM-powered classification (Claude API or other)
 - TREAT framework support
 - Real-world brief or non-law-school writing contexts
 - `.pdf` export
 - Multi-document batch processing
 - User accounts or saved sessions
 - Word processor plugin (e.g., Word add-in)
-

14. Acceptance Criteria

The v1.0 build is complete when:

1. The app launches locally via `python run.py` with no external internet connection required
 2. All three input methods (paste, `.docx`, `.pdf`) successfully produce classification output
 3. IRAC and CREAC framework modes produce correct label sets and colors
 4. Sentence-level highlights and paragraph-level badges both render in the UI
 5. All five paragraph problem types (Missing Rule, Missing Application, Missing Conclusion, Blend, Application Without Rule) are correctly detected and flagged on a test document
 6. Suggestions are displayed for all flagged paragraphs
 7. The annotated `.docx` export downloads successfully and contains highlighted text
 8. The summary bar displays correct aggregate statistics
 9. No API calls are made at any point during normal operation
 10. The app runs without errors on Python 3.10+ on macOS and Windows
-

End of Specification — v1.0